第四章 词法分析2024

王俊彪

C2 词法分析

源程序 → **词法分析** → 词法记号串、错误信息

- 通过扫描源程序逐一识别出每个词法单元。
- 源程序是一个符号串，字母表一般为ASICII字符集；
- 词法记号串也是一个符号串，字母表是？每个词法记号都是某个词法单元的符号表示。
- 从词法分析原理反映出如何拆分源程序符号串、如何识别为词法单元、如何表示为词法记号。
- 能够手写或自动生成词法分析器。

C2.1 词法分析原理

- 源程序是大字符集ASCII上的一个符号串w。
- 词法单元是w的子串x及其所属的语言。
- 子串x所属的语言都是已知的，即根据源语言而设计好的。

2	3	4		*		(	1	1		+	-	2	2	)							

NUM(234) MUL LPAR NUM(11) ADD NUM(-22) RPAR

- 词法分析任务：确定源程序w中每一个子串x及其所属语言。
- 示例确定出7个子串：234，*，（，11，+，-22，）
- 所属语言5个：NUM，MUL，LPR，ADD，RPR

C语言的例子

```
while (i <= 100) {
        s += a[i]; /* form vector total */
        i++;
}
```

- while (reserved word), (, i (identifier), <=, 100 (integer constant),), {, s (identifier), +=, a (identifier), [, i (identifier),], ;, i (identifier), ++, ;, }

- 12个语言：reserved word；（；identifier；<=；integer constant；）；{；+=；[；]；++；;。
- 18个词法单元。
- 空白与注解起分隔记号的作用，词法分析忽略它，而不会识别为记号。
- 双字符记号<=、+=、++不要与单字符记号<、=、+混淆。
- 识别标识符的DFA也识别保留字，但所有保留字是已知的，故可区分。

记号的二元组表示

```
while (i <= 100) {
        s += a[i]; /* form vector total */
        i++;
}
```

while (i <= 100) {\n\ts += a[i]; /* form vector total */\n\ti++;\n}

(WHILE,-) ((,-) (ID,i) (<=,-) (INT,100) (),-) ({,-) (ID,s) (+=,-) (ID,a) ([,-) (ID,i) (],-) (;,-) (ID,i) (++,-) (;,-) (},-)

- 输入是ASCII字符串，输出是记号串（或称单词符号串）
- 记号是类别和值组成的二元组，类别是语言的名字，由于所有语言都是已知的，所以可命名。特别地，单元素语言可以用元素指代它。

(WHILE,-) (LPA,-) (ID,i) (LE,-) (NUM,100) (RPA,-) (LBR,-) (ID,s) (AAS,-) (ID,a) (LBK,-) (ID,i) (RBK,-) (SCO,-) (ID,i) (AAA,-) (SCO,-) (RBR,-)

- 值是子串的表示，可以是子串名、或子串值、或存储位置。
- 比如(NUM,100)中的100可以是数值100也可是字符串“100”
- 比如(ID,i)中i可以是”i”也可以是i在符号表中的登记项。

记号的设计

```
while (i <= 100) {
        s += a[i]; /* form vector total */
        i++;
}
```

(WHILE,-) (LPA,-) (ID,i) (LE,-) (NUM,100) (RPA,-) (LBR,-) (ID,s) (AAS,-) (ID,a) (LBK,-) (ID,i) (RBK,-) (SCO,-) (ID,i) (AAA,-) (SCO,-) (RBR,-)

- 全体一种：把某个符号串集合命名为语言LNAME，它的每个元素是一个类别为LNAME的词法单元。如ID，INT
- 一符一种：单元素语言的词法单元只有类别，为语言名字，值省略。
- 只能设计为全体一种的语言：元素可有近无穷个。比如数、标识符等
- 对于有穷语言的记号设计，两种都可以，取决于效果。
  - 关键字 (WHILE,-) (KEY, while)
  - 运算符 (LE,-) (ROP,<=)
  - 括号 (LBR,-) (BRACE, [)

常见记号的类别

类别	类别名字	语言定义
Identifiers	ID	[a–zA–Z_][a–zA–Z_0–9]*
Integer constants	INT	[+-]?[0–9]+
Character constants	CHR	'([a–zA–Z0–9]|\[a–zA–Z])'
Floating constants	FLOAT	[+-]?[0-9]*. [0-9][0-9]*|[+-]?[0-9][0-9]*[. [0-9]*]?
String constants	STRING	"([a–zA–Z0–9]|\[a–zA–Z])*"
Operator tokens	ADD, MUL, …	+, *, <,<=,==, ∧, ∨, ¬, …
Assignments	ASG, AASG, …	=, +=, ++, …
括号	LPAR, LBK, …	(,), [,], {, }, ..
Keywords	IF, INT, …	while, int, if, else, return, …

对源程序预处理

- 该忽略的忽略，该处理的处理，因为它们不是记号。

类别	*Examples*
comment	/* ignored */
preprocessor directive	#include <foo.h>
	#define NUMS 5, 6
macro	NUMS
whitespace	\t \n \b

回顾C程序

<源程序>→(<文本行>\n | <文本行><注释行>\n | <注释行>\n)+<EOF>

<文本行>→<预处理命令行>

<预处理命令行>→<宏定义> | <文件包含>| <编译指示>

<宏定义>→#define<不含\n的串>

<文件包含>→#include<不含\n的串>

<编译指示>→#pragma<不含\n的串>

<续行>→<文本行>\\

<文本行>→<续行>

<文本行>→ (<分隔> <有效串>)$^+$| <有效串>(<分隔> <有效串>)*

<分隔>→(<注释块> | <空白>)$^+$

<注释行>→//<不含\n串>

<注释块>→/*<不含*/串>*/

<空白>→(\t | \r |<空格>)$^+$

<有效串>→(r_1 | … | r_n)$^+$

预处理结果

<源程序>→(<有效串><分隔符>)$^+$<EOF>

<有效串>→ $(r_1 \mid \ldots \mid r_n)^+$

<分隔符>可以是任意感兴趣的且不在U1≤i≤n · Σ_i中的符号。

扫描一个<有效串>返回一个记号串，直到遇到<EOF>，依次将各记号串连接作为词法分析结果。

针对有效串的拆分与识别

逐一扫描<有效串>#直到EOF结束。

<有效串>→$(r_1 \mid \ldots \mid r_n)^+$

return21# ➔ (ID, 'r)(ID, 'eturn21)×

return21# ➔ (return, _)(NUM 21)×

return21# ➔ (ID, 'return21)√

21return# ➔ (NUM, 21)(RETURN, _)√

2+3# ➔ (NUM, 2)(ADD, _)(NUM, 3)?√

2+3# ➔ (NUM, 2)(NUM, +3)?×

+15.07 ➔ (ADD, _)(NUM, 15)(FLO, .07)×

+15.07 ➔ (NUM, +15)(FLO, .07)×

+15.07 ➔ (ADD, _)(FLO, 15.07)?√

+15.07 ➔ (FLO, +15.07)?√

+N15.07 ➔ (ADD, _)(ID, 'N)(FLO, 15.07)×

+N15.07 ➔ (ADD, _)(ID, 'N15)(FLO, .07)√

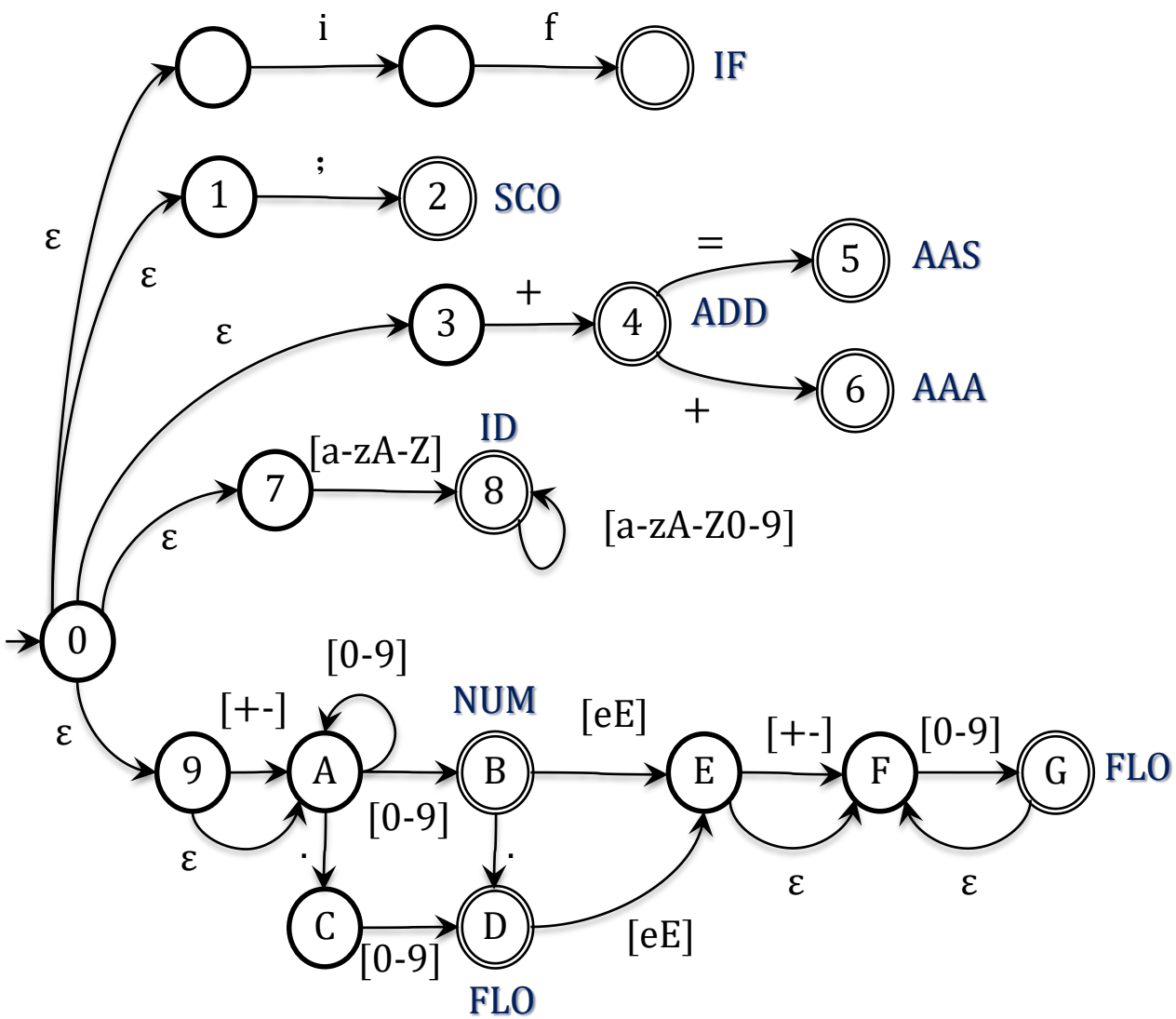
一些情形可能不在实际语言中出现；但实际语言服从此原理。这为学生打开了思维空间

- 要解决的问题：
- ①剩余<有效串>的每一个前缀就是一个拆分方案，哪一个才是词法单元？
  - *前缀最大化原则*
- ②拆分出的前缀可能属于多个语言，其类别选哪个？
  - *事实优先级*
- ③当前缀被识别为词法单元后，表示为一个词法记号，接着从剩余串中继续①和②直到剩余串为#结束。
- ④输出的记号串与<有效串>保持次序一致。可并行加速。

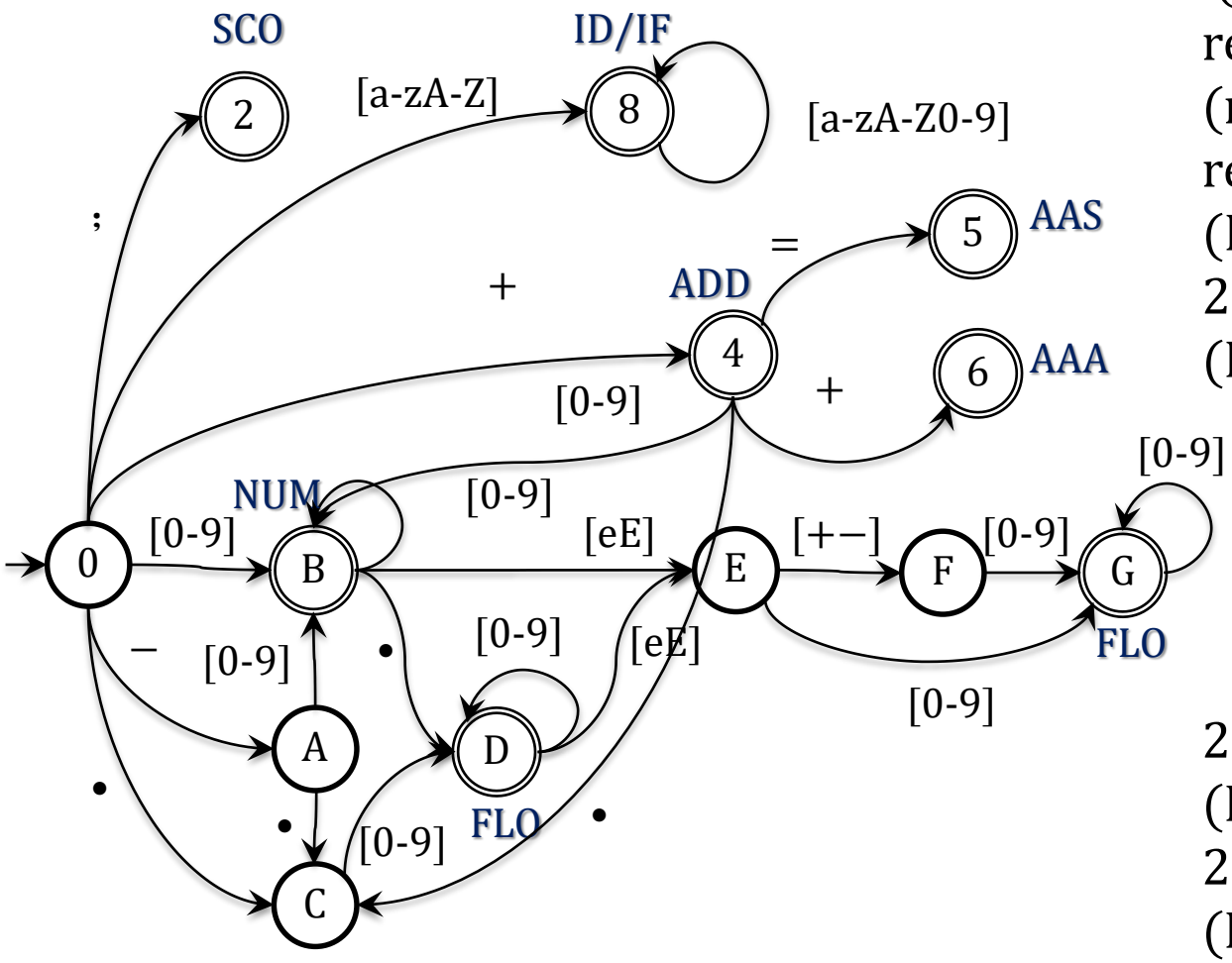
多语言联合自动机识别原理

- 假定有n个语言依次定义为$r_1, \ldots, r_n$
- <有效串>$\in L((r_1|\ldots|r_n)^+)$
- 为每个r_i构建DFA A_i，初始状态为q_i。每个接受状态都带有标记L_i。
- 构建多语言联合NFA，从初始状态q_0都有ε弧射出到q_i。
- 确定化为一个多语言联合DFA，是σ_DFA。 σ_DFA的每个接受状态的标记这样确定，如果它包含多个NFA接受状态，那么选优先级最高的那个语言作为标记
- 采用事实优先级，即下标较小者优先级较高。

例


i
f
IF
;
1
2
SCO
ε
=
5
AAS
+
3
4
ADD
6
AAA
+
[a-zA-Z]
ID
7
8
[a-zA-Z0-9]
0
[0-9]
[+-]
NUM
[eE]
[+-]
[0-9]
9
A
B
E
F
G
FLO
[0-9]
.
.
C
D
[0-9]
FLO
[eE]

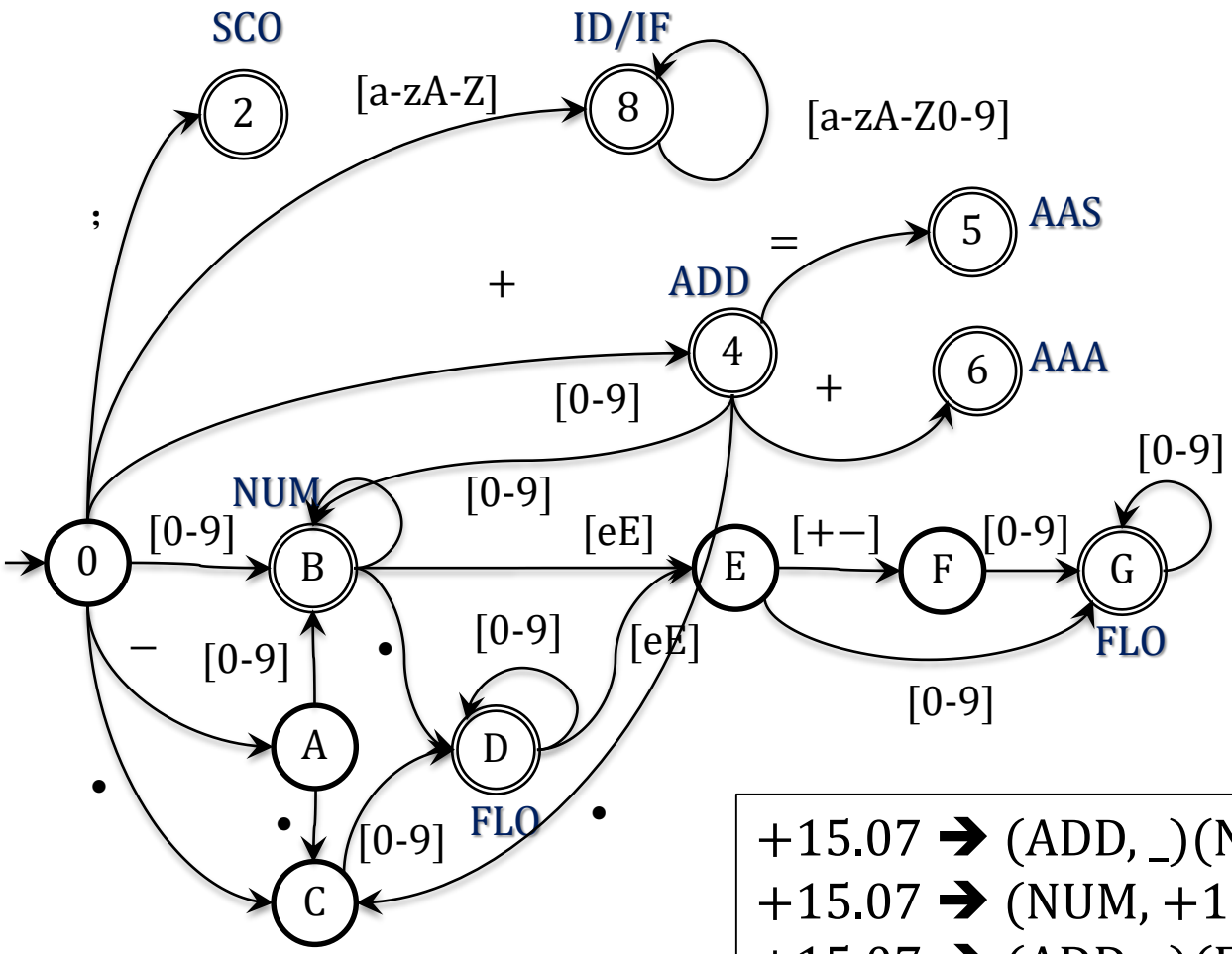
σ_DFA的识别举例



return21# ➔
(ID, 'r)(ID, 'eturn21)✕
return21# ➔
(return, _)(NUM 21)✕
return21# ➔
(ID, 'return21)√
21return# ➔
(NUM, 21)(RETURN, _)√

2+3# ➔
(NUM, 2)(ADD, _)(NUM, 3)?√
2+3# ➔
(NUM, 2)(NUM, +3)?✕

σ_DFA的识别举例



+15.07 ➔ (ADD, _)(NUM, 15)(FLO, .07)×
+15.07 ➔ (NUM, +15)(FLO, .07)×
+15.07 ➔ (ADD, _)(FLO, 15.07)?√
+15.07 ➔ (FLO, +15.07)?√
+N15.07 ➔ (ADD, _)(ID, 'N)(FLO, 15.07)×
+N15.07 ➔ (ADD, _)(ID, 'N15)(FLO, .07)√